

Cloud-Agnostic Infrastructure Automation

Author: Dhanush V

Role: MCA Graduate | Cloud & DevOps

GitHub Repository: [git@github.com:dhanushv4/cloud-agnostics.git](https://github.com/dhanushv4/cloud-agnostics.git)

Project Overview

This project automates cloud infrastructure deployment using Terraform and Ansible while supporting multiple cloud providers through a single configuration file.

Instead of manually creating virtual machines, security groups, and software installations, the project generates Terraform files automatically, deploys infrastructure, configures servers using Ansible, verifies deployment, and cleans up resources when no longer required.

The objective is to demonstrate Infrastructure as Code (IaC), automation, and cloud provisioning skills suitable for Cloud and DevOps Engineer roles.

Objectives

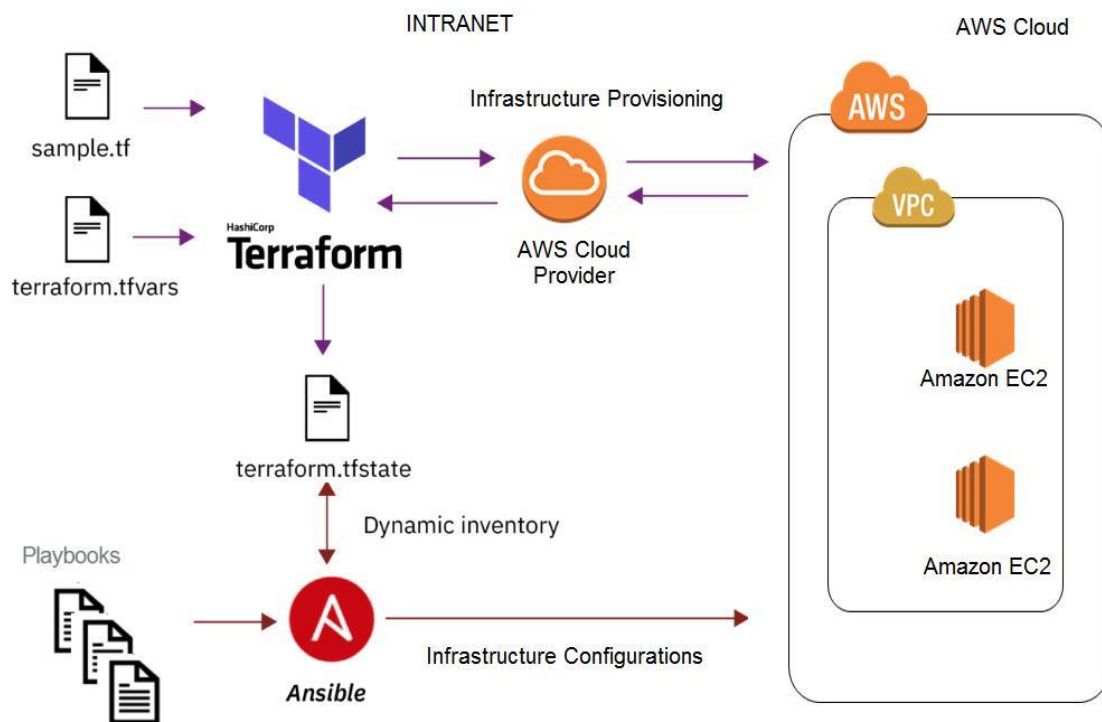
- Automate infrastructure provisioning.
- Support multiple cloud providers.
- Use Infrastructure as Code.
- Configure servers automatically.
- Reduce manual deployment effort.
- Demonstrate DevOps automation workflow.

Technologies Used

Category	Technology
Cloud	AWS
IaC	Terraform
Configuration	Ansible
OS	Ubuntu Linux
Scripting	Bash
Version Control	Git

Repository GitHub
SSH OpenSSH

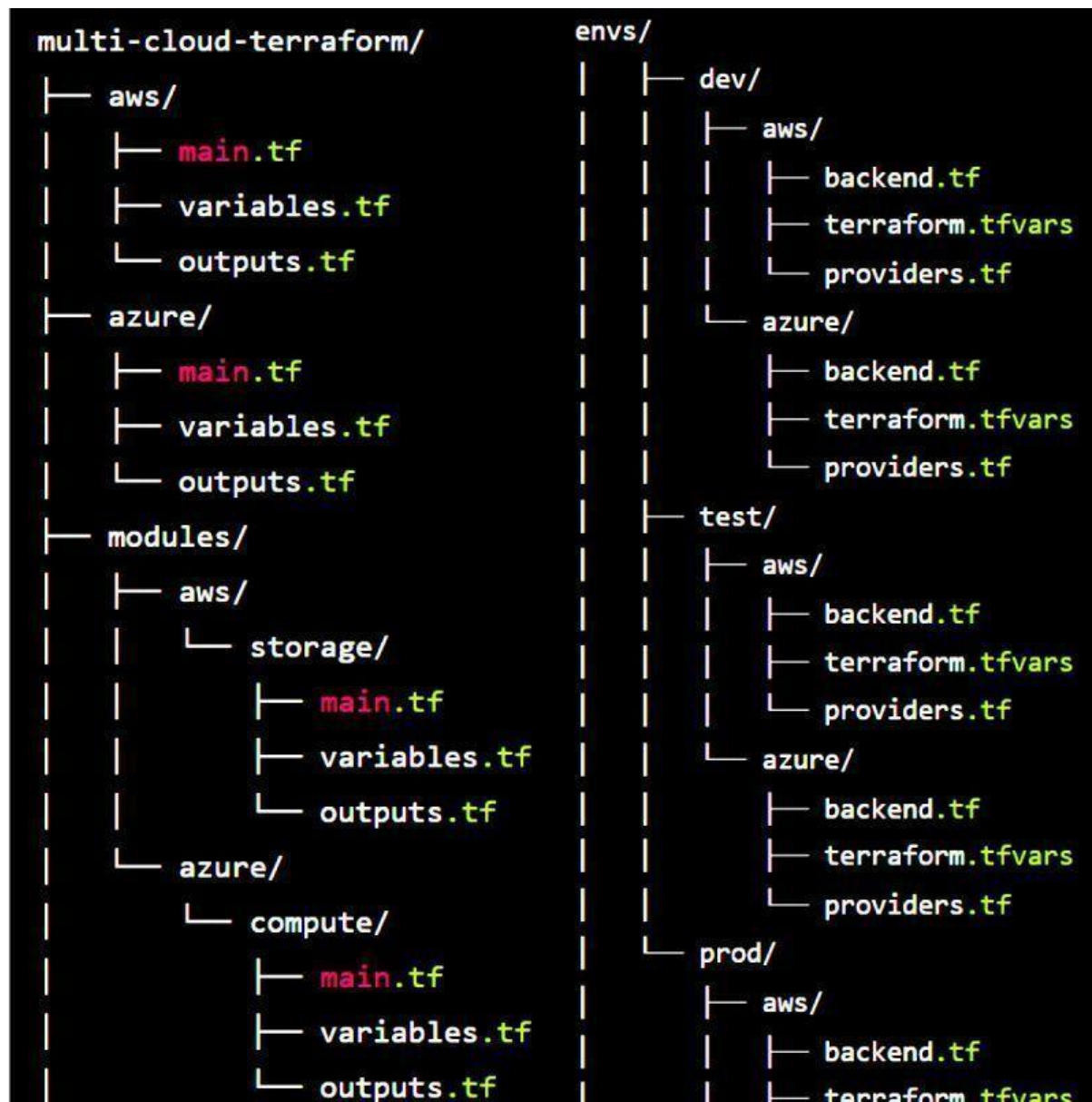
Project Architecture



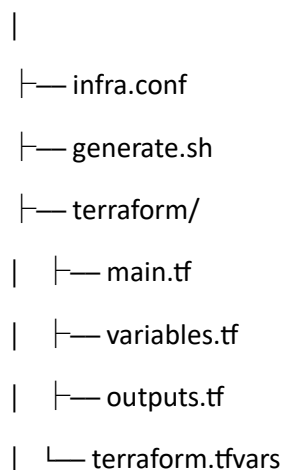
Workflow

1. User edits `infra.conf`.
2. Bash script reads configuration.
3. Terraform files are generated.
4. Terraform provisions infrastructure.
5. EC2 instance is created.
6. Security Group is configured.
7. Ansible connects through SSH.
8. Required packages are installed.
9. Validation scripts verify deployment.
10. Infrastructure can be destroyed automatically.

Project Structure



cloud-agnostics/



```
|
| └─ ansible/
|   └─ docker.yml
|   └─ nginx.yml
|   └─ cleanup.yml
|   └─ validate.yml
|
| └─ scripts/
|   └─ terraform-init.sh
|   └─ terraform-apply.sh
|   └─ verify.sh
|   └─ destroy.sh
|
└─ README.md
```

Configuration File

The project is controlled through a single configuration file named `infra.conf`.

Example:

```
CLOUD_PROVIDER="aws"
```

```
AWS_REGION="ap-south-1"
```

```
INSTANCE_COUNT="1"
```

```
INSTANCE_TYPE="t2.micro"
```

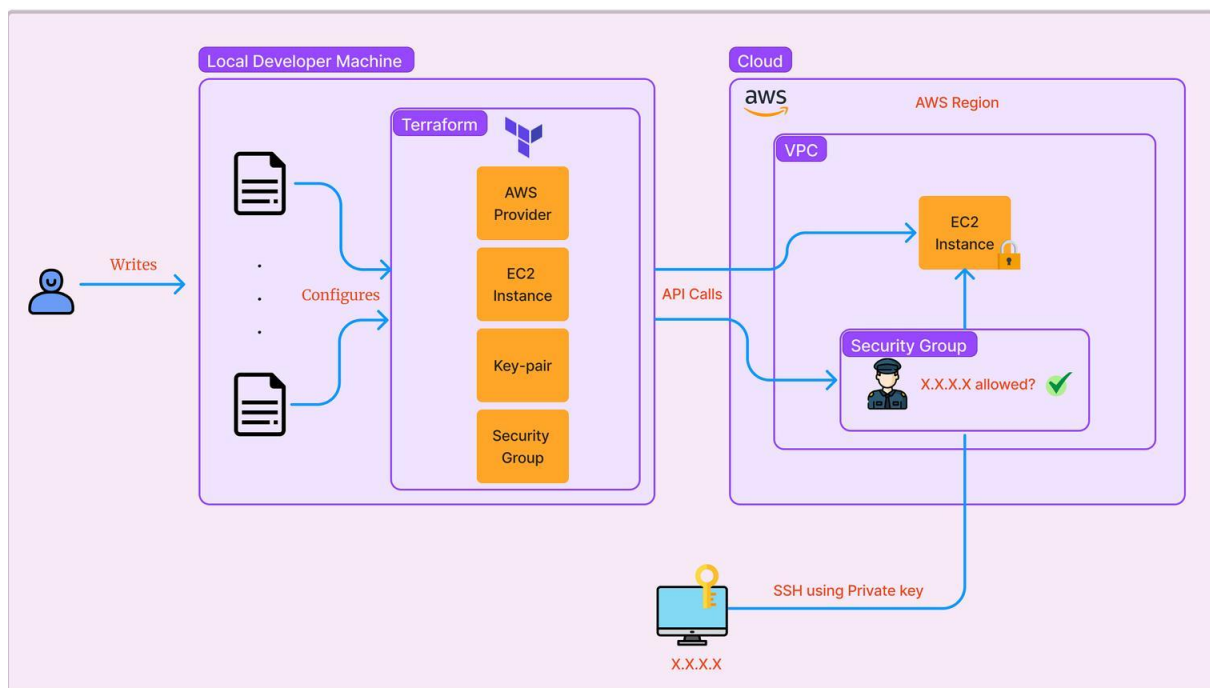
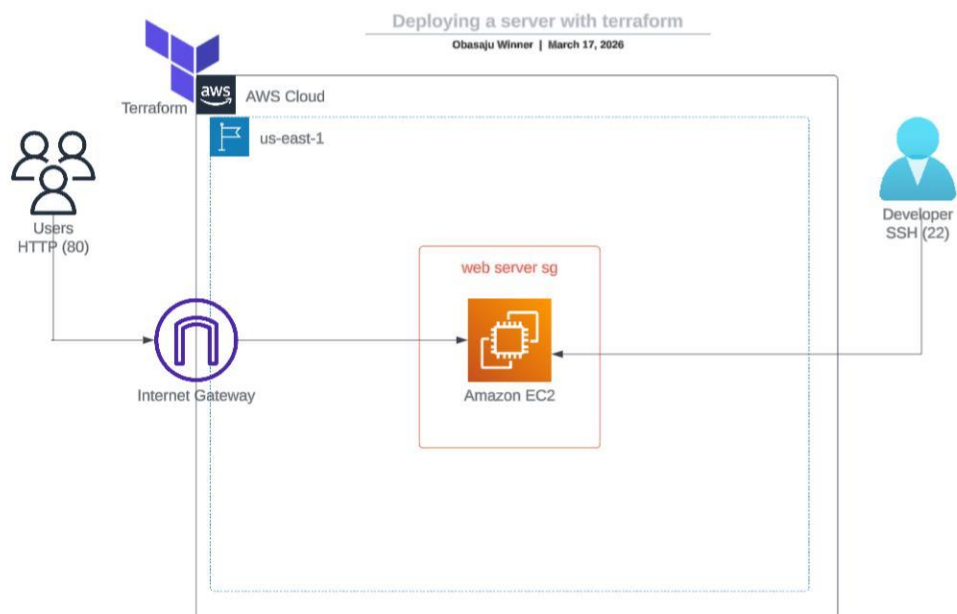
```
TAG_NAME="cloud-demo"
```

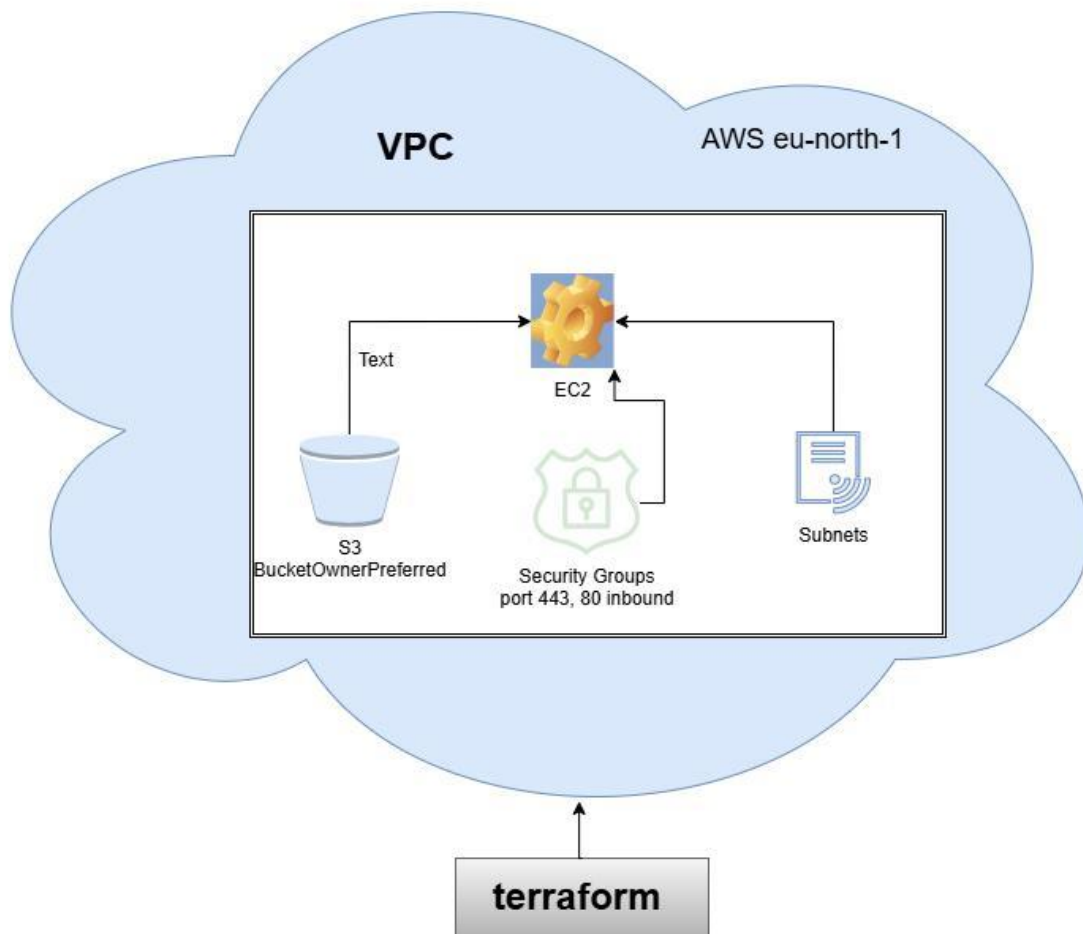
```
SSH_USER="ubuntu"
```

```
SECURITY_GROUP="cloud-demo-sg"
```

This allows infrastructure settings to be changed without modifying Terraform code.

Terraform Implementation





Terraform performs the following tasks:

- Creates EC2 instances.
- Creates Security Groups.
- Opens SSH access.
- Tags cloud resources.
- Produces outputs such as Public IP.

Example command:

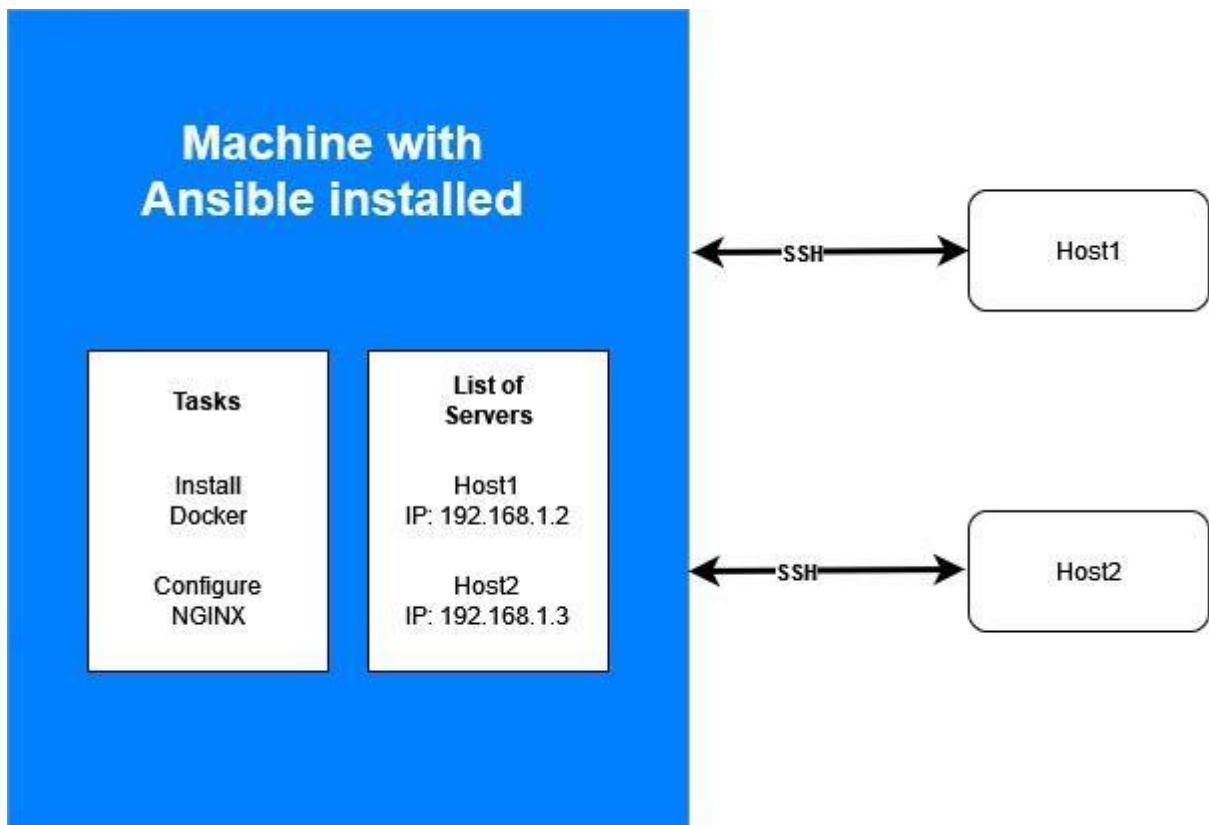
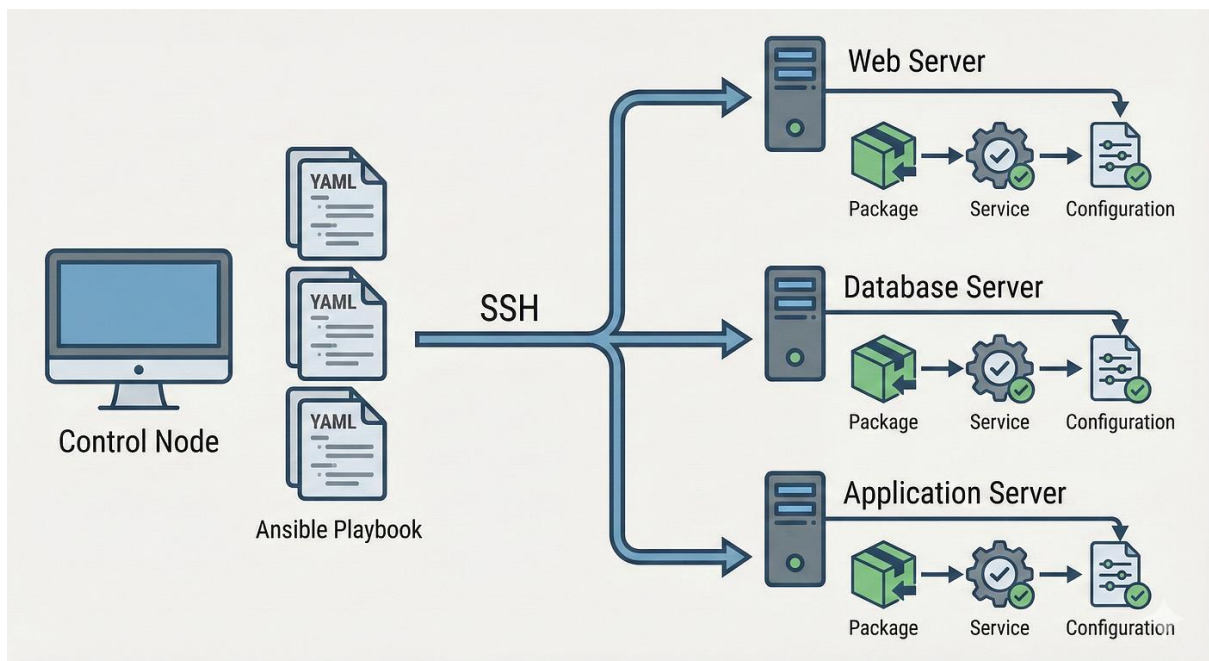
```
terraform init
```

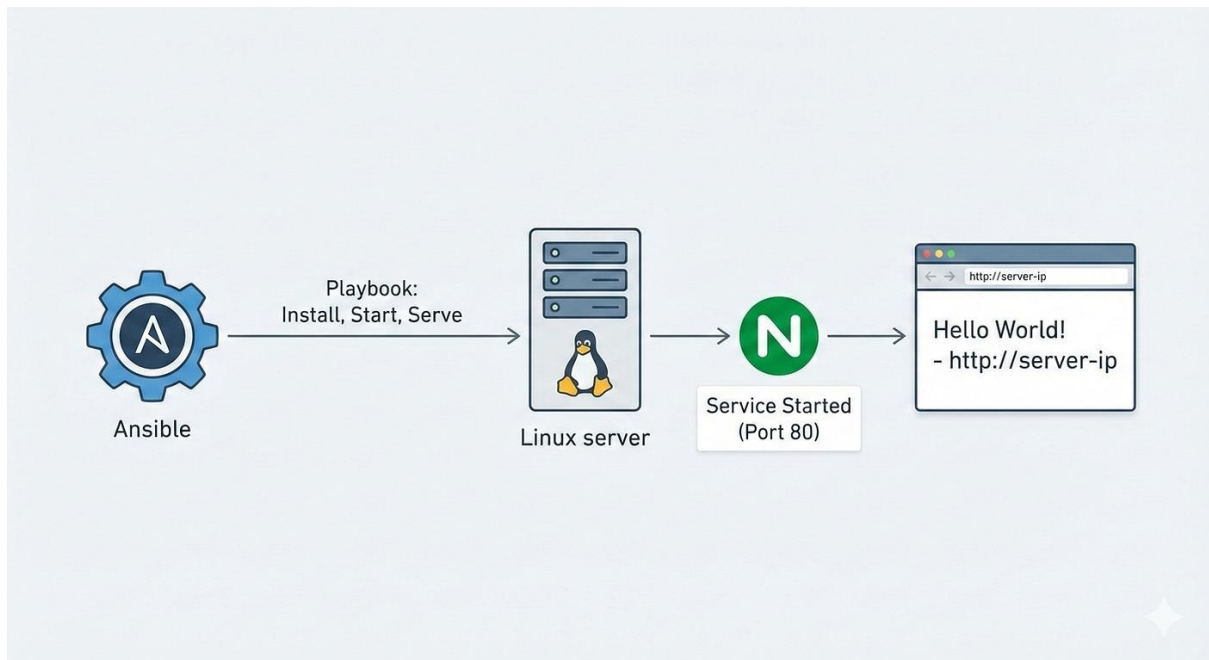
```
terraform apply
```

Outputs include:

- Public IP
- Instance ID
- SSH connection details

Ansible Automation





After Terraform creates the server, Ansible automatically configures it.

Playbooks include:

- Docker installation
- NGINX installation
- Validation
- Cleanup

Example:

```
ansible-playbook docker.yml
```

```
ansible-playbook nginx.yml
```

Deployment Process

Step 1

Configure infrastructure.

```
nano infra.conf
```

Step 2

Generate Terraform files.

```
./generate.sh
```

Step 3

Initialize Terraform.

```
terraform init
```

Step 4

Deploy infrastructure.

terraform apply

Step 5

Connect through SSH.

ssh ubuntu@PUBLIC_IP

Step 6

Run Ansible playbooks.

ansible-playbook nginx.yml

Step 7

Validate deployment.

./verify.sh

Security Configuration

The project follows basic cloud security practices.

Implemented:

- Security Groups
- SSH access control
- Ubuntu user login
- Infrastructure tagging
- Automated cleanup

Verification

Deployment verification includes:

- SSH connectivity
- NGINX status
- Docker installation
- Running services
- Server accessibility

Example:

systemctl status nginx

docker --version

Cleanup Process

Infrastructure can be removed completely.

terraform destroy

This deletes:

- EC2 instances
- Security Groups
- Associated cloud resources

Key Features

- Infrastructure as Code using Terraform
- Automated server configuration using Ansible
- Single configuration file
- Cloud-ready deployment workflow
- Automated validation
- Automated cleanup
- Reusable deployment process

Challenges Faced

During development several issues were resolved:

- Terraform variable configuration
- SSH connectivity
- Security Group permissions
- Ansible inventory generation
- Infrastructure validation
- Automated deployment workflow

These improvements made the deployment process more reliable.

Skills Demonstrated

Cloud

- AWS EC2
- Security Groups
- Infrastructure deployment

DevOps

- Terraform
- Ansible
- Git
- Bash
- Linux
- SSH

Automation

- Infrastructure provisioning
- Server configuration
- Validation scripts
- Resource cleanup